一、数据探索

通过读取 student_marks.csv，可以发现其包含 56 行和 13 列。其中，共存在 13 个字段，通过 Student_ID 字段来识别每一个学生，通过 Test_1 至 Test_12 提现每个学生的 12 次测试成绩。通过 Excel 检查可以发现，没有重复的 Student_ID，并且从 Test_1 到 Test_10 所有字段下，学生的成绩都分布在 0 到 100 分之间，意味着没有逻辑错误的数据。

下面的表 1 则列出了读取得到的，来自原始数据集中，前五名学生（前五行）的十二次成绩，表 2 则列出了 Test_1 到 Test_12 共计 12 次考试中，所有 56 名学生成绩的平均数、标准差、最小值和最大值。

表 1　原始数据集前 5 行数据

字段	第 1 行	第 2 行	第 3 行	第 4 行	第 5 行
Student_ID	22000	22001	22002	22003	22004
Test_1	78	79	66	60	99
Test_2	87	71	65	58	95
Test_3	91	81	70	54	96
Test_4	91	72	74	61	93
Test_5	88	73	78	54	97
Test_6	98	68	86	57	89
Test_7	94	59	87	64	92
Test_8	100	69	96	62	98
Test_9	100	59	88	72	91
Test_10	100	60	82	63	98
Test_11	100	61	90	72	95
Test_12	93	67	86	76	88

表 2　每次考试的平均数、标准差、最小值和最大值

字段	平均数	标准差	最小值	最大值
Test_1	70.75	17.01	40	100
Test_2	69.20	17.71	34	100
Test_3	68.09	18.84	35	100
Test_4	67.45	19.81	28	100
Test_5	67.30	20.75	26	100
Test_6	66.00	21.05	29	100
Test_7	66.16	21.43	26	100
Test_8	65.30	22.73	19	100
Test_9	64.39	23.21	9	100
Test_10	64.25	22.60	12	100
Test_11	64.52	22.61	19	100
Test_12	65.93	22.46	20	100

然后，以每次考试的学生成绩作为指标，计算了 12 次考试之间成绩的皮尔逊相关系数矩阵，然后绘制了如下图 1 的相关系数的热力图。可以很容易观察到，各个格子的颜色都基本上是红色，意味着各场考试都是正相关的，并且相邻考试之间，学生成绩的相关性比不相邻的更高。

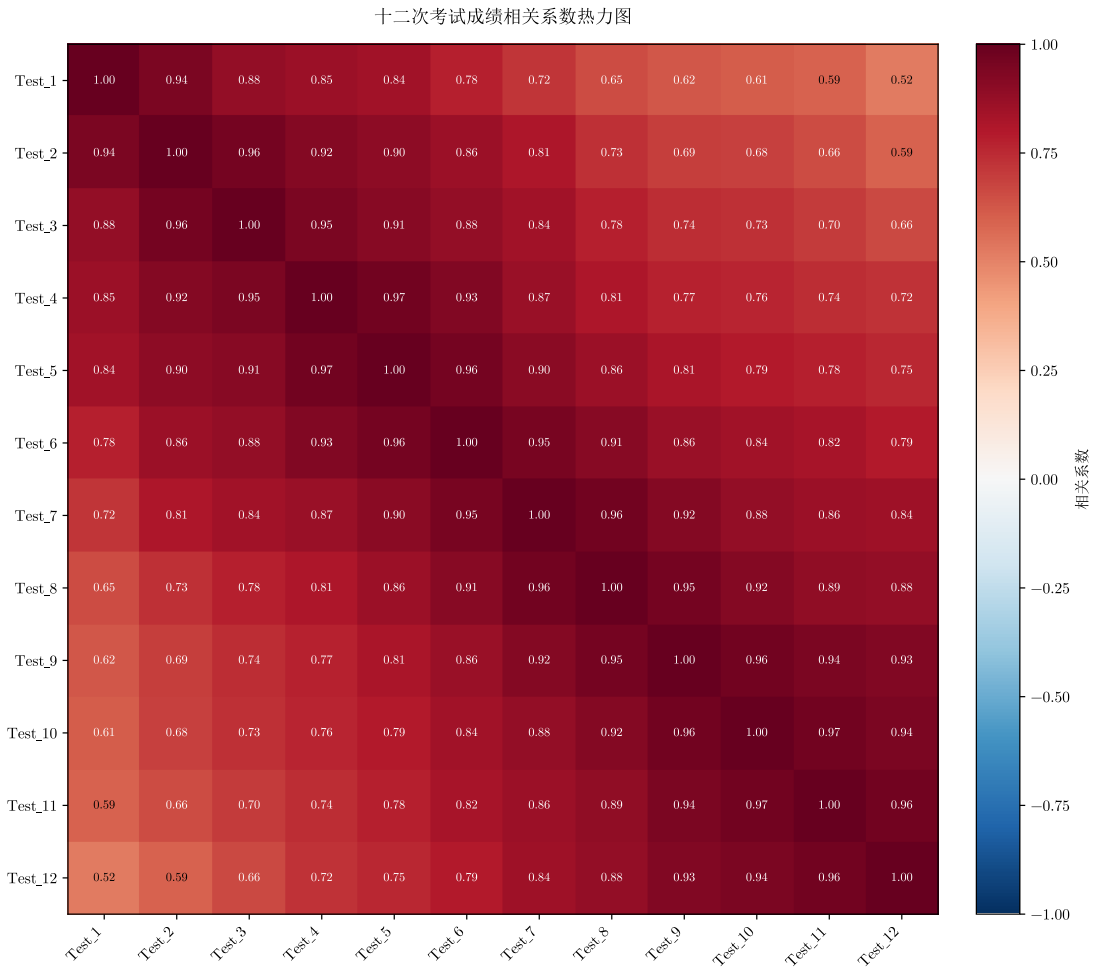


图 1　十二次考试成绩相关系数热力图

从数据读取到描述性分析，再到相关系数的计算，对应的关键代码如下代码 1 所示。

代码 1　　数据读取、描述性统计和相关系数统计

```
df = load_data()
scores = df.drop(columns="Student_ID")  # 学号不参与成绩分析，可以丢弃
description = scores.describe().T[["mean", "std", "min", "max"]]

# 先计算 Pearson 相关系数，再绘制热力图
correlation = scores.corr()
plot_correlation(correlation, Config.FIGURES / "成绩相关系数热力图.svg")
```

对于问题一，通过计算得到的成绩相关矩阵，与 Test_12 相关性最高的特征是 Test_11，对应的相关系数约为 0.96，其次是 Test_10 和 Test_9，对应的相关系数约为 0.94 和 0.93。另外，我们发现，随着第一次考试到第十一次考试，各场考试成绩与第十二次考试成绩的相关性系数变得越来越大。这个现象可能意味着，最近几场的考试可以更好的用于反映和解释学生当前的水平，而对于越早的考试，解释的力度可能相对减弱，但这并不意味着更早的

考试无法在预测未来成绩上提供 0 贡献，还需要进一步的分析。

二、建立基线模型

现在我们来制作基线的线性回归模型来预测学生们未来一场考试的成绩。在建立基线模型之前，我们考虑从数据集中随机选取 80%的样本作为训练集，剩余 20%的样本作为测试集。在设置随机种子 42 的情况下，这样的方式选出了 44 名学生作为训练集，剩余的 12 人则作为测试集。

为了确保不同模型之间可以公平地进行比较，通过采用如下代码 2 的数据划分方式。输入为前 11 次考试成绩，预测目标为 Test_12。在固定随机种子后，各模型使用相同的训练学生和测试学生。

代码 2　　划分数据集

```
# 按学生划分数据: 80% 用于训练, 20%用于测试
x_train, x_test, y_train, y_test = train_test_split(
    df[Config.FEATURES], df[Config.TARGET], test_size=0.2,
random_state=Config.SEED
)
```

首先，对于基线 A（均值基线），不使用任何特征，直接以训练集 Test_12 的均值作为对所有学生的预测，即使用平均成绩 65.5682 作为预测值，对应了一个常数函数 $\widehat{\text{Test}_{12}} = 65.5682$。将测试集的成绩代入计算可得 RMSE 约为 22.43，R^2 约为 -0.00，意味着使用平均成绩作为预测值，意味着这种预测方式没办法看出不同学生之间，成绩的高低和不同，将会掩盖差异。

然后，对于基线 B（近因基线），通过使用 Test_11 作为输入特征，做一元线性回归，去预测 Test_12 的值，在训练集上得到线性回归函数为：

$$\widehat{\text{Test}_{12}} = 0.9702 \times \text{Test}_{11} + 3.1236$$

将测试集的真实成绩与这个一元线性回归的预测成绩计算，得到 MSE 约为 24.59，RMSE 约为 4.96，R^2 约为 0.95。与均值基线算法相比，我们的均方根误差值 RMSE 从 22.43 下降到了 4.96，下降幅度很大，并且 R^2 也从 -0.00 提高到了 0.95，说明最近一次的考试成绩可以更好地预测和解释第十二次成绩，而均值基线算法却不能很好发挥作用。

对于核心代码，两种基线的核心实现如下代码 3 所示。均值基线使用训练集 Test_12 的平均成绩作为预测值，而近因基线仅使用 Test_11 作为特征，在训练集上拟合后进行预测。两个模型都通过测试集中的真实成绩计算和评估评价模型的性能。

代码 3　　基线模型的拟合

```
# 基线A: 所有测试学生都预测为训练集均分
mean_prediction = np.full(len(y_test), y_train.mean())
mean_metrics = evaluate(y_test, mean_prediction)
```

```
# 基线 B: 只用 Test_11, 在训练集拟合后预测测试集
recent = LinearRegression().fit(x_train[["Test_11"]], y_train)
recent_prediction = recent.predict(x_test[["Test_11"]])
recent_metrics = evaluate(y_test, recent_prediction)
```

对于问题二，均值基线的 RMSE 和 Test_12 的标准差接近可以从它们的计算公式来进行理解。其中，均值基线的 RMSE 的平方可以表示为下面的算式。

$$\mathrm{RMSE}^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - C)^2\,, \qquad C = \overline{y_{tr}}$$

其中，y_i 指示测试集中的第 i 名学生的成绩，n 是测试集的成绩数量，C 是这个均值基线算法在训练集取的成绩平均值 $\overline{y_{tr}}$，因为是个常数，这里用 C 来表示以更好理解。

另外，由于：

$$\overline{y_{te}} = \frac{\sum_{i=1}^{n} y_i}{n}$$

即：

$$n \times \overline{y_{te}} = \sum_{i=1}^{n} y_i$$

则可以很容易导出：

$$\sum_{i=1}^{n}(y_i - \overline{y_{te}}) = \sum_{i=1}^{n} y_i - \sum_{i=1}^{n}\overline{y_{te}} = \left(\sum_{i=1}^{n} y_i\right) - (n \times \overline{y_{te}}) = \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} y_i = 0$$

也即：

$$\sum_{i=1}^{n} y_i - \sum_{i=1}^{n} y_i = 0 \qquad (2-1)$$

其中，$\overline{y_{te}}$ 是测试数据集上学生成绩的平均值。于是，RMSE 的平方可以改写为下式：

$$\mathrm{RMSE}^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - C)^2$$

$$= \frac{1}{n}\sum_{i=1}^{n}[(y_i - \overline{y_{te}}) + (\overline{y_{te}} - C)]^2$$

$$= \left[\frac{1}{n}\sum_{i=1}^{n}(y_i - \overline{y_{te}})^2\right] + \left[\frac{2}{n}\sum_{i=1}^{n}(y_i - \overline{y_{te}})(\overline{y_{te}} - C)\right] + \left[\frac{1}{n}\sum_{i=1}^{n}(\overline{y_{te}} - C)^2\right]$$

$$= \left\{\frac{1}{n}\left[\sum_{i=1}^{n}(y_i - \overline{y_{te}})^2 + (\overline{y_{te}} - C)^2\right]\right\} + \left[\frac{2}{n} \times (\overline{y_{te}} - C) \times \sum_{i=1}^{n}(y_i - \overline{y_{te}})\right]$$

根据式 $2-1$ 可将 $\sum_{i=1}^{n}(y_i - \overline{y_{te}})$ 替代为 0，则有：

$$= \left\{\frac{1}{n}\left[\sum_{i=1}^{n}(y_i - \overline{y_{te}})^2 + (\overline{y_{te}} - C)^2\right]\right\} + \left[\frac{2}{n} \times (\overline{y_{te}} - C) \times 0\right]$$

$$= \frac{1}{n}\left[\sum_{i=1}^{n}(y_i - \overline{y_{te}})^2 + (\overline{y_{te}} - C)^2\right]$$

也即：

$$\mathrm{RMSE}^2 = \frac{1}{n}\left[\sum_{i=1}^{n}(y_i - \overline{y_{te}})^2 + \sum_{i=1}^{n}(\overline{y_{te}} - C)^2\right] \qquad (2-2)$$

其中，第一项为测试集 Test_12 成绩的总体方差，第二项为测试集与训练集平均成绩之差的平方。因此，当两个数据集的平均成绩接近时，第二项较小，均值基线的测试 RMSE 就近似等于测试集 Test_12 的总体标准差。这也同时意味着，当训练集和测试集的平均成绩完全相同时，二者恰好相等。

三、多元线性回归建模

按照与第二问相同的方式，用 train_test_split 将数据集划分出 80%的训练集和 20%的测试集。这里也仍然使用 42 作为随机种子。欲拟合多元线性回归模型，我们可以规定预测公式满足：

$$\widehat{y_i} = \alpha + \sum_{j=1}^{11}\beta_j x_{ij}$$

其中，$\widehat{y_i}$ 指示第 i 个学生的第十二次成绩的预测值，β_j 指示线性回归对于第 j 次考试贡献的权重，x_{ij} 指示学生 i 在第 j 场考试的真实成绩，α 是这个多元线性回归模型的截距。应注意的是，对于任意的学生 i，我们每次计算预测值时都使用一组相同的权重 β_j 和截距 α，意味着我们并非对每位学生进行线性回归拟合，而是一次性对所有学生进行线性回归拟合。

在 80%的训练集上拟合该模型，对应的代码如下代码 4 所示。通过使成本函数 MSE 最小化，拟合出的线性回归模型所使用的各个权重参数和截距参数（精确到小数点后两位）如下表 3 所示。

代码 4　　多元线性回归的拟合和评估

```python
# 使用 44 名训练学生的全部 11 个特征拟合模型
model = LinearRegression().fit(x_train, y_train)

train_prediction = model.predict(x_train)
test_prediction = model.predict(x_test)

# 分别评价训练集和测试集
train_metrics = evaluate(y_train, train_prediction)
test_metrics = evaluate(y_test, test_prediction)
```

表 3　多元线性回归拟合结果

参数	值	参数	值
α	7.23	β_6	−0.38
β_1	−0.06	β_7	0.44
β_2	−0.49	β_8	−0.21
β_3	0.09	β_9	0.17
β_4	0.59	β_{10}	0.04
β_5	−0.08	β_{11}	0.80

为了评估模型的性能，分别在 80%的训练集和 20%的测试集上预测各个学生的第 12 次考试成绩，计算所得的均方误差 MSE、均方根误差 RMSE、决定系数 R^2 如下表 4 所示。

表 4　训练集和测试集的性能表现

数据集	MSE	RMSE	R^2
训练集（80%）	22.03	4.69	0.96
测试集（20%）	26.34	5.13	0.94

可以看到测试集的性能相较训练集的有略微下降，因为 MSE 和 RMSE 都提高了，说明模型的预测误差增加了，而 R^2 下降了，意味着模型的可解释力度下降了。发生这种情况的原因也很容易理解和解释，模型对于从未见过的数据（测试集），性能可能不如训练的时候好。就像是每天做模拟题一样，在训练结束后即便可以完美的过完所有模拟题，也不一定能在真正的考试中取得相同或者更好的性能，情况可能会比模拟题的更差一点。然而，只要数据量充足（对应做了很多模拟题，身经百战），模型在未见数据的性能上也差不到哪里去。

我们以第12次考试的真实成绩作为 x 轴，对应的预测成绩作为 y 轴，收集每名学生的 (真实成绩, 预测成绩) 并绘制成散点图如下图 2 所示。显而易见的是，当真实成绩和预测成绩相等时完全预测正确，此时是 $y = x$ 的一条正比例函数。肉眼上观察这个散点图，无论是训练集和测试集，几乎所有的点都靠近 $y = x$ 这条线，说明几乎所有预测都很接近真实值。当然，也有少量的点距离 $y = x$ 较远，有一些差异，但总体上可以认为模型的性能较好。

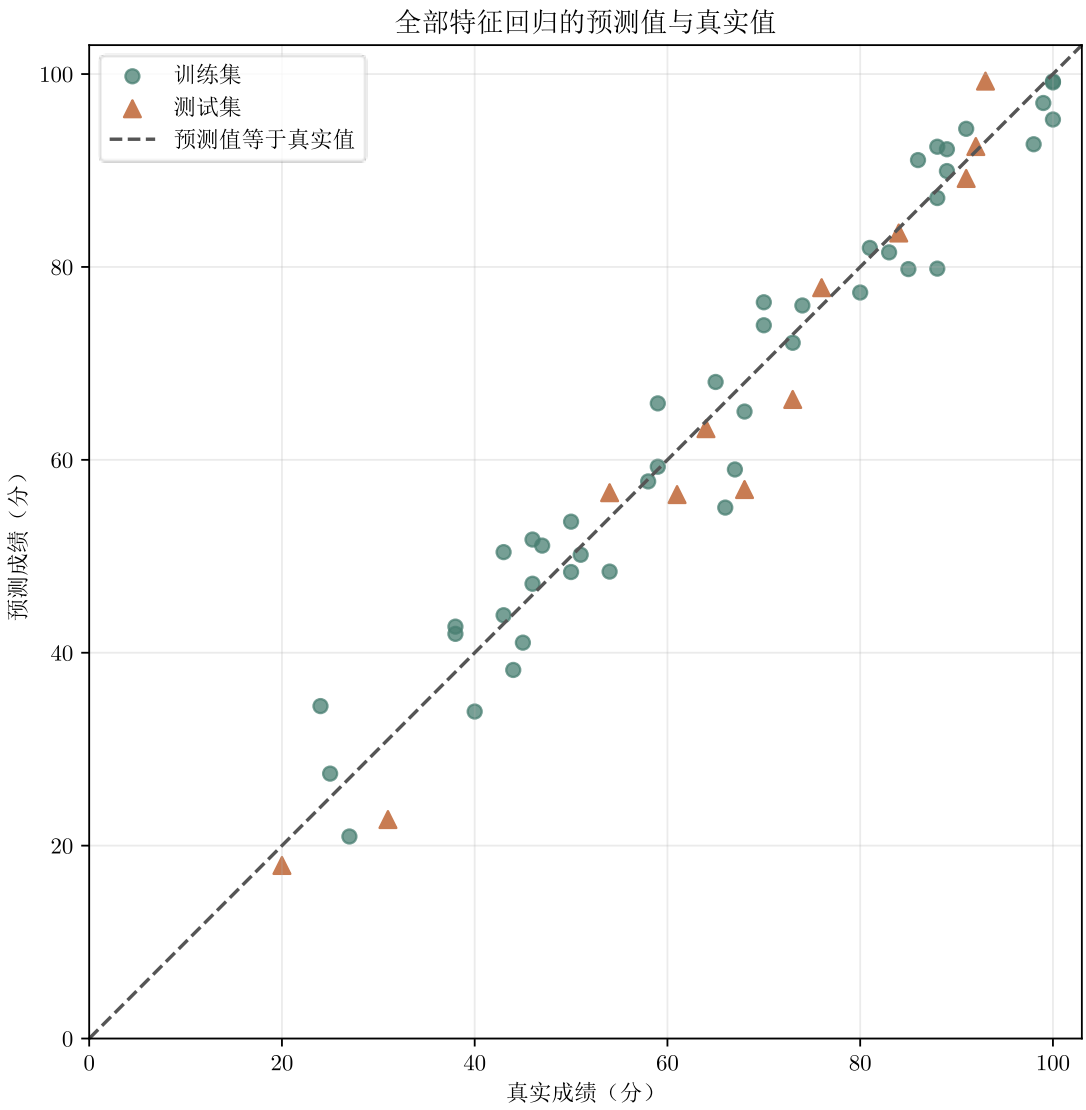


图 2　预测成绩 VS 真实成绩

下表 5 则列出了当前的多元线性回归模型和基线 B 的单元线性回归模型的性能比较，同时给出了在数据集和在测试集上的 MSE、RMSE 和 R^2 值。

表 5　当前模型和基线 B 模型的性能比较

模型	**数据集**	**MSE**	**RMSE**	$\boldsymbol{R^2}$
11 元回归	训练集	22.03	4.69	0.96
基线 B	训练集	37.28	6.11	0.92
11 元回归	测试集	26.34	5.13	0.94
基线 B	测试集	24.59	4.96	0.95

对于问题三，可以发现，当前模型的性能 MSE、RMSE 相比于基线 B 在训练集上是下降的，R^2 则是上升的。然而，当前模型的 MSE、RMSE 在测试集上则提高了，并且 R^2 上还下降了。

这一结果和我多用了 10 个特征的预期不太一致。因为，我们一般认为，更早的成绩虽然对未来成绩的贡献较小，但如果我们考虑了这样的因素，这应该比只使用最近一次成绩作为预测贡献了更多的信息（熵），模型的效果应该更好。

然而，我们在测试集上却观察到了更差的性能和解释性，意味着当前的训练集和测试集的划分并没有改善模型的预测。虽然当前模型在训练集上表现的比基线 B 模型更好，但这种效果没有延续到模型未见的测试集上。

这意味着我的经验法则（提供更早的成绩用于预测可以提高模型的性能和鲁棒性）不一定总是正确的，但这也可能是数据集的划分问题，可能还需要进一步研究原因。

一个合理的解释是，早期的学生成绩和近期的学生成绩有一些重复的信息，将所有过往的成绩都考虑进来，会增加模型的波动性。

四、回归诊断与深入解读

4.1 多重共线性诊断

我们通过 VIF 指标来判断多元线性回归的共线性程度，即，我们要判断某个特征是否能被其他特征较好的线性解释。在本文中，特征是过去的考试成绩。对于某个特征 j，我们固定其他特征的值，然后以这些固定特征作为自变量，目标特征 j 作为因变量，进行线性回归拟合。然后，这个线性回归的 R^2 就是对于 j 的决定系数 R_j^2，于是我们就可以通过下面的算式来计算该特征的 VIF_j 的值。

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

同时易得：

$$R_j^2 = 1 - \frac{1}{\mathrm{VIF}_j}$$

在第三部分的基础上，我们使用相同的随机种子 42，并继续使用相同的训练集和测试集，在训练集的 44 名学生的 11 次考试成绩上运行 VIF。

在实现上，我们在训练数据中引入全为 1 的常数列，使辅助回归能够包含截距。随后逐列计算 Test_1 到 Test_11 一共 11 个特征的 VIF，对应的核心代码如下代码 5 所示，每个特征对应的 VIF 值如下表 6 所示。

代码 5　　11 个特征的 VIF 值计算

```
design = sm.add_constant(x_train)  # 添加全为 1 的列，用于估计截距
model = sm.OLS(y_train, design).fit()
```

```
# 从第 1 列开始，跳过第 0 列的常数项
vif = [
    variance_inflation_factor(design.values, index)
    for index in range(1, design.shape[1])
]

coefficients = model.params.drop("const")
target_correlation = x_train.corrwith(y_train)
```

表 6　各特征在训练集上的 VIF 值

特征	VIF	特征	VIF
Test_1	8.66	Test_2	27.24
Test_3	17.98	Test_4	27.77
Test_5	27.35	Test_6	23.85
Test_7	30.27	Test_8	27.14
Test_9	24.71	Test_10	22.15
Test_11	16.03	/	/

可以看到，Test_1 具有最小的 VIF 值，为 8.66，而 Test_7 的 VIF 值是最大的，为 30.27。对于其他特征，大多数特征都分布在 20 以上，只有 Test3 和 Test_11 是介于 10 到 20 之间的（含）。

对于问题四，之前的热力图是偏深红的，意味着不同考试成绩之间存在着明显的正相关关系，部分特征的相关系数超过了 0.9，并且相邻考试之间的相关系数相较其他的更加地突出。根据之前的观察结果，热力图可以揭示一些问题，即，学生的不同场考试之间有较强的连续性，并且 Test_1 到 Test_11 之间的特征不是相互独立的，每场考试的成绩都携带了大量的重复信息。

VIF 的结果则更进一步地验证了这一点。Test_1 的 VIF 作为 8.66 已经非常接近 10 了，并且其他特征的 VIF 都超过 10，且 Test_7 的达到 30 多，是最大的，说明从 Test_1 开始共线性就非常明显了，并且 Test_7 的 30 多 VIF 可以说明，在训练集中，其他非 Test_7 的特征组合起来可以很好的解释 Test_7，因而 Test_7 和其他特征存在大量的重复贡献。另外，在工程上，我们一般认为，对多个 VIF 大于 10 的特征而言，这些特征存在着较强的共线性，意味着这些特征在解释目标变量的增加或减少时，都提供了较多的重复信息（熵）。

特征的共线性会影响模型的系数稳定性和模型的解释力度。具体来说，对于几个高度相关的变量，模型无法很好的分辨这些变量的独立作用（贡献），这会导致这些变量在预测时都提供重复的类似的贡献，而掩盖过其他特征能贡献的信息，导致模型的预测误差和标准差增大，模型具有较高的波动性。另外，这些共线性的变量，最终算出来的值加起来后，可能挤占了其他特征能用的空间，从而导致共线性的情况下，其他特征得是负贡献才能使线性回归的 Cost Function 的值最小，然而正常情况下这些特征应该是正的贡献，这意味着特征的权重系数的正负号发生了反转，可能会得到“认为枪杀案的增多可以促进人们买房一样”的荒谬模型，在解释上完全站不住脚。

4.2 回归系数解读

表 7　回归系数为负而相关系数为正的特征

特征	与 Test_12 的训练集相关系数	回归系数
Test_1	0.45	−0.06
Test_2	0.53	−0.49
Test_5	0.74	−0.08
Test_6	0.75	−0.38
Test_8	0.86	−0.21

对于问题五，上表 7 呈现了多元线性回归在拟合完成后，回归系数为负但相关系数为正的特征。与 Test_12 特征的相关系数为正意味着，在该场考试取得较好考试成绩的学生，理所应当在第 12 次测试中得到好的成绩。然而，回归系数为负意味着，在第三部分的多元线性回归模型中，在保持其他 10 次考试成绩不变的情况下，这场考试的成绩每多得 1 分，最终的第 12 次考试成绩却下降了。

对于“控制其他变量”的含义，以第 2 场考试为例，它与 Test_12 的相关系数为正，意味着在第 2 场考试取得好成绩者，在第 12 次测试也应该取得较好的成绩。但负的回归系数 −0.49 意味着，如果不改变其他 10 场考试成绩，只让这名学生在这一第 2 场考试取得更高的成绩，那么每高 1 分，就意味着第 12 次成绩的预测结果少 0.49 分。

这个正负的差异是不矛盾的，因为相关系数侧重于这个特征和 Test_12 之间的关系，而回归系数是放在一个多元回归模型的，它还得看其他场次考试对模型的贡献，所以也不一定是正的。不过，由于从 Test_1 到 Test_11 之间总共 11 个特征的共线性很强，也可能是因为共线性反转了表 7 中这些系数的正负号。

因此，负数的系数在存在多个变量影响，并且共线性较强的情况下，不能简单的认为负的系数就是说这个特征是负贡献的。因为我们的成本函数只是所有预测误差的平方和加起来然后平均一下，所以模型可能把更多的空间分配到了其他特征上，从而导致 Test_2 等特征必须以负系数出现，才能获得 Cost Function 的最小值。

4.3 残差诊断

为了检查第三部分得到的多元线性回归模型的适用性，我们绘制了残差与拟合值的散点图和残差 QQ 图。如下的代码 6 以训练集为例，展示了残差散点图和 Q-Q 图的关键绘制过程。对于测试集也仍采用相同的方法，本处不再赘述。

代码 6　　散点图和 QQ 图绘制部分的关键代码（训练集）

```
# 正残差表示模型低估成绩
residual = y_train - train_prediction
```

```python
# 残差与拟合值散点图
plt.subplot(1, 2, 1)
plt.scatter(train_prediction, residual)
plt.axhline(0, linestyle="--")

# Q-Q 图: 将有序残差与正态分布的理论分位数比较
(theoretical, ordered), (slope, intercept, _) =
stats.probplot(residual, dist="norm")
plt.subplot(1, 2, 2)
plt.scatter(theoretical, ordered)
plt.plot(theoretical, slope * theoretical + intercept)
```

如下图 3 则是绘制的图像。对于左上角和左下角的两张图，我们分别以 (训练集拟合成绩, 训练集残差) 和 (测试集拟合成绩, 测试集残差) 绘制了对应的散点图。对于右上角和右下角的图像，则分别是训练集的 QQ 图和测试集的 QQ 图。

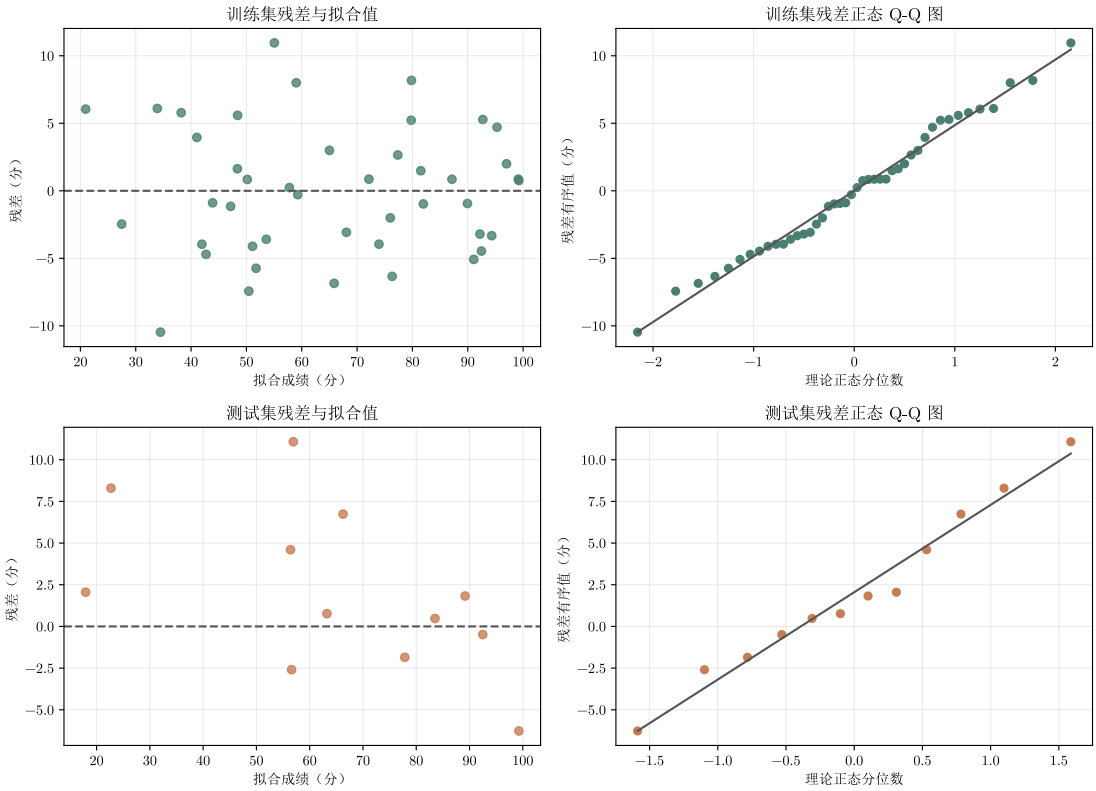


图 3　残差与拟合值的散点图和残差 QQ 图

可以看到，训练集散点图中残差大致分布在 0 的两侧，正残差和负残差各有 22 个，最小值大概是 -10 分，最大值大概是 $+10$ 分，整体分布是相对比较对称的。对于训练集的 QQ 图，里面大多数点都是沿参考线分布的，只有很少量的点存在一定的偏离，意味着残差分布似乎是正态分布的形状。

对于问题六，从以上观察到的结果可以得出，残差基本上是对称分布在 0 附近的，并且散点图没有明显的弯曲迹象，也没有喇叭形的形状，说明并

没有非线性和异方差的问题。因此，可以认为线性假设、同方差假设和正态性假设是满足的。不过，这个只是基于图 3 的肉眼观察初步得出的。后续可能需要实施严格的正态性检验和 BP 检验来确定正态性假设和同方差假设是否满足。

4.4 核心结论

用于预测第 12 次成绩的信息主要来自于距离它的最近几场考试，其中，第 11 场考试的贡献是最大的，它与第 12 次成绩的相关系数达到 0.96。

本文分布讨论了基线 B 模型和考虑了所有 11 个特征多元回归模型的性能，并发现基线 B 的 RMSE 为 4.9589，而多元线性是 5.1327，意味着在随机种子为 42，80%为训练集，20%划分为测试集的情况下，多元线性回归表现更差，也说明了考虑早期的成绩不能使模型的性能得到提升。

本文也在 4.2 节计算了第 1 次考试到第 11 次考试中，各个特征的 VIF 值，并发现仅有 1 个特征的 VIF 值小于 10，其他全部大于 10，并且大多数特征的 VIF 值都大于 20，最高的是 30 多。这意味着模型很难区分各个特征对最终考试成绩的贡献，这些特征之间有很强的共线性关系，回归系数容易随样本变化而波动，导致我们无法根据系数的正负和大小简单地判断某次考试对第 12 次测试的成绩的实际影响。

五、模型改进

与 Test_12 这个特征相关系数最高的 3 个特征分别是 Test11、Test_10 和 Test_9。现在，以这三个特征构建三元线性回归模型如下所示。其中，$\widehat{y_i}'$ 是该线性回归模型对学生 i 在第 12 场成绩的新预测值，α' 是该 3 元线性回归的截距，β'_{11}、β'_{10} 和 β'_9 分别对应第 11、第 10 和第 9 次考试成绩的系数，x_{i11}、x_{i10} 和 x_{i9} 分别是该学生 i 与第 11、第 10 和第 9 次考试取得的真实成绩。

$$\widehat{y_i}' = \alpha' + \beta'_{11}x_{i11} + \beta'_{10}x_{i10} + \beta'_9 x_{i9}$$

特征选择仅使用训练集完成，避免测试集信息参与模型构建。按照各特征与 Test_12 的相关系数绝对值选出前三名，重新拟合线性回归，并在原来的测试集上进行评价，对应的核心代码如下代码 7 所示。

代码 7　　特征选择、3 元线性回归的拟合和评估

```
# 仅使用训练集选出相关性最高的三个特征
selected =
x_train.corrwith(y_train).abs().nlargest(3).index.tolist()
model = LinearRegression().fit(x_train[selected], y_train)

# 保持原测试集不变，评价特征选择后的模型
test_prediction = model.predict(x_test[selected])
test_metrics = evaluate(y_test, test_prediction)
```

因此，在相同的随机种子 42，80%划分为训练集，20%划分为测试集的数据集划分方法下，特征选择的结果是选中了 Test_11、Test_10 和 Test_9。在训练集上重新拟合线性回归后，该 3 元线性回归模型的各个参数的训练结果（精确到小数点后两位）如下表 8 所示。

表 8　多元线性回归拟合结果

参数	值
α'	2.07
β'_{11}	0.79
β'_{10}	0.02
β'_{9}	0.17

然后我们计算了拟合模型在测试集上的性能，得到了对应的 MSE、RMSE 和 R^2 值，并与全 11 个特征的 11 元多元线性回归模型和基线 B 模型的性能进行了比较，结果如下表 9 所示。

表 9　多个模型的预测性能比较

模型	特征数	MSE	RMSE	R^2
全特征	11	26.34	5.13	0.9473
本模型	3	24.74	4.97	0.9505
基线 B	1	24.59	4.96	0.9508

可以看到，基线 B 模型仍然在指标方面仍然是表现最优的，它的 MSE 和 RMSE 是三者之中最小的，R^2 是三者之中最大的。虽然本模型相较 11 元的全特征线性回归存在性能提升（MSE 和 RMSE 下降且 R^2 提高），但效果仍然没有基线 B 模型的好。然而，全特征回归模型和本模型距离基线 B 模型的性能差距都不大，不一定说明它一定始终更好。

六、结果呈现

表 10　　所有模型的性能比较

模型	特征数	训练 RMSE	测试 RMSE	测试 R^2
基线 A	0	22.22	22.43	−0.0057
基线 B	1	6.11	4.96	0.9508
11 元回归	11	4.69	5.13	0.9473
改进模型	3	5.97	4.97	0.9505

由上表 10，均值基线（基线 A）的测试误差较大，无法反映学生之间的成绩差异。加入考试成绩后，不再采用平均值计算，而是用线性回归模型，各回归模型的预测效果得到了明显改善。

全 11 特征的 11 元线性回归模型的训练误差最低，但测试表现略差于基线 B 模型和经过特征选择后的 3 元线性回归模型，意味着增加特征没有在随

机数为 42 的 8:2 比例的划分下得到更好的预测效果。

特征选择过程中，我们选取了与第 12 次考试成绩最相关的第 9、第 10 和第 11 次考试成绩做 3 元线性回归，测试 RMSE 从 11 元线性回归的 5.13 下降至 4.97。基线 B 的测试 RMSE 最低，为 4.96，且仅使用一个特征，说明最近一次考试已经包含较强的预测信息。

然而，基线 B 与三个特征模型的测试误差十分接近，且测试集仅有 12 人，因此无法很好的分辨基线 B、11 元线性回归模型和特征选择后的 3 元线性回归模型的好坏。

七、数据解读与分析

对于问题一的回答，详见问题一回答。
对于问题二的回答，详见问题二回答。
对于问题三的回答，详见问题三回答。
对于问题四的回答，详见问题四回答。
对于问题五的回答，详见问题五回答。
对于问题六的回答，详见问题六回答。

八、AI 工具使用声明

本人未与他人讨论本实验，但使用了 OpenAI Codex 的 GPT 6.1 Sol 模型辅助代码编写与修改，也包括图表绘制、结果分析和报告文字的草拟。在这基础上，再经过大量人工修改、复核和排版后撰写出本文。

九、附录

`analysis.py` (程序总入口)

```python
import pandas as pd
from step_one_exploration import run_step_one
from step_two_baselines import run_step_two
from step_three_regression import run_step_three
from step_four_diagnostics import run_step_four
from step_five_improvement import run_step_five

def run_experiment() -> None:
    """运行全部"""
    run_step_one()
    baselines = run_step_two()
    _, regression = run_step_three()
    run_step_four()
    report = pd.concat([baselines, regression], ignore_index=True)
    run_step_five(report)

run_experiment()
```

`common.py` (读取数据、数据集划分和指标计算)

```python
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

class Config:
    ROOT: Path = Path(__file__).resolve().parent
    FIGURES: Path = ROOT / "figures"
    FEATURES: list[str] = [f"Test_{number}" for number in range(1,
12)]
    TARGET: str = "Test_12"
    SEED: int = 42
    METRIC_COLUMNS: list[str] = [
        "模型",
        "特征数",
        "训练 MSE",
        "训练 RMSE",
        "训练 R²",
        "测试 MSE",
        "测试 RMSE",
        "测试 R²",
    ]

def load_data() -> pd.DataFrame:
    """加载数据"""
    Config.FIGURES.mkdir(exist_ok=True)
    return pd.read_csv(Config.ROOT / "student_marks.csv")

def split_data() -> tuple[pd.DataFrame, pd.DataFrame, pd.Series,
pd.Series]:
    """数据集拆分"""
    df = load_data()
    return train_test_split(
        df[Config.FEATURES], df[Config.TARGET], test_size=0.2,
random_state=Config.SEED
    )

def evaluate(y: pd.Series, prediction: np.ndarray) -> list[float]:
    """计算 MSE, RMSE 和 R²"""
    mse = mean_squared_error(y, prediction)
    return [mse, np.sqrt(mse), r2_score(y, prediction)]
```

plot.py (绘制分析图表)

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from typing import Any
from scipy import stats
from pathlib import Path

class PlotConfig:
    STYLE: dict[str, Any] = {
        "font.family": ["Latin Modern Math", "SimSun"],
```

```
        "mathtext.fontset": "cm",
        "axes.formatter.use_mathtext": True,
        "axes.unicode_minus": False,
        "svg.fonttype": "path",
    }
    TRAIN_COLOR: str = "#497f72"
    TEST_COLOR: str = "#c87c53"

def plot_correlation(correlation: pd.DataFrame, output_path: Path)
-> None:
    """展示十二次考试成绩之间的相关性"""
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(10, 9))
        plt.imshow(correlation, cmap="RdBu_r", vmin=-1, vmax=1)
        plt.colorbar(label="相关系数", fraction=0.046, pad=0.04)
        positions = range(len(correlation))
        plt.xticks(positions, correlation.columns, rotation=45,
ha="right")
        plt.yticks(positions, correlation.columns)
        for row in positions:
            for column in positions:
                value = correlation.iloc[row, column]
                color = "white" if abs(value) >= 0.6 else "black"
                plt.text(
                    column,
                    row,
                    f"{value:.2f}",
                    ha="center",
                    va="center",
                    fontsize=8,
                    color=color,
                )
        plt.title("十二次考试成绩相关系数热力图", pad=14)
        plt.tight_layout()
        plt.savefig(output_path, format="svg", bbox_inches="tight")
        plt.close()

def plot_predictions(
    y_train: pd.Series,
    y_test: pd.Series,
    train_prediction: np.ndarray,
    test_prediction: np.ndarray,
    output_path: Path,
) -> None:
    """区分训练和测试记录，观察多元回归的预测误差"""
    limit = max(100, train_prediction.max(), test_prediction.max())
+ 3
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(8, 7))
        plt.scatter(
            y_train,
            train_prediction,
            color=PlotConfig.TRAIN_COLOR,
            alpha=0.75,
            s=38,
            label="训练集",
```

```
        )
        plt.scatter(
            y_test,
            test_prediction,
            color=PlotConfig.TEST_COLOR,
            marker="^",
            s=55,
            label="测试集",
        )
        plt.plot(
            [0, limit], [0, limit], "--", color="#555555", label="预
测值等于真实值"
        )
        plt.xlim(0, limit)
        plt.ylim(0, limit)
        plt.xlabel("真实成绩（分）")
        plt.ylabel("预测成绩（分）")
        plt.title("全部特征回归的预测值与真实值")
        plt.legend()
        plt.grid(alpha=0.25)
        plt.gca().set_aspect("equal")
        plt.tight_layout()
        plt.savefig(output_path, format="svg", bbox_inches="tight")
        plt.close()

def plot_residuals(
    y_train: pd.Series,
    y_test: pd.Series,
    train_prediction: np.ndarray,
    test_prediction: np.ndarray,
    output_path: Path,
) -> None:
    """用残差散点图与正态 Q-Q 图检查训练和测试误差"""
    groups = [
        (
            "训练集",
            y_train - train_prediction,
            train_prediction,
            PlotConfig.TRAIN_COLOR,
        ),
        ("测试集", y_test - test_prediction, test_prediction,
PlotConfig.TEST_COLOR),
    ]
    with plt.rc_context(PlotConfig.STYLE):
        plt.figure(figsize=(11, 8))
        for index, (name, residual, prediction, color) in
enumerate(groups):
            plt.subplot(2, 2, index * 2 + 1)
            plt.scatter(prediction, residual, color=color, s=35,
alpha=0.8)
            plt.axhline(0, color="#555555", linestyle="--")
            plt.title(f"{name}残差与拟合值")
            plt.xlabel("拟合成绩（分）")
            plt.ylabel("残差（分）")
            plt.grid(alpha=0.25)
```

```
            plt.subplot(2, 2, index * 2 + 2)
            (theoretical, ordered), (slope, intercept, _) =
stats.probplot(
                residual, dist="norm"
            )
            plt.scatter(theoretical, ordered, color=color, s=30)
            plt.plot(theoretical, slope * theoretical + intercept,
color="#555555")
            plt.title(f"{name}残差正态 Q-Q 图")
            plt.xlabel("理论正态分位数")
            plt.ylabel("残差有序值（分）")
            plt.grid(alpha=0.25)
        plt.tight_layout()
        plt.savefig(output_path, format="svg", bbox_inches="tight")
        plt.close()
```

step_one_exploration.py (阶段一数据探索)

```
from common import Config, load_data
from plot import plot_correlation

def run_step_one() -> None:
    """数据探索"""
    print("=" * 25)

    df = load_data()
    scores = df.drop(columns="Student_ID")
    description = scores.describe().T[["mean", "std", "min", "max"]]
    description.columns = ["平均数", "标准差", "最小值", "最大值"]

    print(f"数据形状 (行数, 列数): {df.shape}")
    print("-" * 5)
    print("前五行: \n" + df.head().to_string(index=False))
    print("-" * 5)
    print("描述统计: \n" + description.round(2).to_string())
    print("-" * 5)
    print(f"缺失值: {df.isna().sum().sum()}")
    print(f"越界成绩: {((scores < 0) | (scores > 100)).sum().sum()}")

    correlation = scores.corr()
    plot_correlation(correlation, Config.FIGURES / "成绩相关系数热力
图.svg")
    target_correlation = correlation.loc[Config.FEATURES,
Config.TARGET]
    best = target_correlation.idxmax()
    print(
        f"T1: 与 Test_12 相关性最高的是 {best}, 相关系数为
{target_correlation[best]:.4f}"
    )
```

step_two_baselines.py (阶段二建立基线模型)

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from common import Config, evaluate, split_data
```

```python
def run_step_two() -> pd.DataFrame:
    """建立基线模型"""
    print("=" * 25)
    x_train, x_test, y_train, y_test = split_data()
    recent = LinearRegression().fit(x_train[["Test_11"]], y_train)
    rows = [
        [
            "均值基线",
            0,
            *evaluate(y_train, np.full(len(y_train),
y_train.mean())),
            *evaluate(y_test, np.full(len(y_test), y_train.mean())),
        ],
        [
            "Test_11 回归",
            1,
            *evaluate(y_train, recent.predict(x_train[["Test_11"]])),
            *evaluate(y_test, recent.predict(x_test[["Test_11"]])),
        ],
    ]
    print(f"训练/测试人数: {len(y_train)}/{len(y_test)}, 随机种子:
{Config.SEED}")
    print(f"训练/测试均值: {y_train.mean():.4f}/{y_test.mean():.4f}分
")
    print(f"测试总体标准差: {y_test.std(ddof=0):.4f}分")
    return pd.DataFrame(rows, columns=Config.METRIC_COLUMNS)
```

step_three_regression.py (阶段三多元线性回归建模)

```python
import pandas as pd
from plot import plot_predictions
from common import Config, evaluate, split_data
from sklearn.linear_model import LinearRegression

def run_step_three() -> tuple[LinearRegression, pd.DataFrame]:
    """多元线性回归建模"""
    x_train, x_test, y_train, y_test = split_data()
    model = LinearRegression().fit(x_train, y_train)
    train_prediction = model.predict(x_train)
    test_prediction = model.predict(x_test)
    row = [
        "全部特征回归",
        len(Config.FEATURES),
        *evaluate(y_train, train_prediction),
        *evaluate(y_test, test_prediction),
    ]
    plot_predictions(
        y_train,
        y_test,
        train_prediction,
        test_prediction,
        Config.FIGURES / "预测成绩与真实成绩.svg",
    )
    return model, pd.DataFrame([row], columns=Config.METRIC_COLUMNS)
```

step_four_diagnostics.py (阶段四回归诊断与深入解读)

```python
import pandas as pd
import statsmodels.api as sm
from plot import plot_residuals
from common import Config, split_data
from statsmodels.stats.outliers_influence import
variance_inflation_factor

def run_step_four() -> pd.DataFrame:
    """回归诊断与深入解读"""
    print("=" * 25)

    x_train, x_test, y_train, y_test = split_data()
    design = sm.add_constant(x_train)
    model = sm.OLS(y_train, design).fit()
    table = pd.DataFrame(
        {
            "回归系数": model.params.drop("const"),
            "VIF": [
                variance_inflation_factor(design.values, index)
                for index in range(1, design.shape[1])
            ],
            "与目标的训练集相关系数": x_train.corrwith(y_train),
        }
    )

    print(f"多元回归截距: {model.params['const']:.4f}")
    print("系数与共线性诊断: \n" + table.round(4).to_string())

    plot_residuals(
        y_train,
        y_test,
        model.fittedvalues.to_numpy(),
        model.predict(sm.add_constant(x_test)).to_numpy(),
        Config.FIGURES / "回归残差诊断.svg",
    )
    return table
```

step_five_improvement.py (阶段五模型改进)

```python
import pandas as pd
from sklearn.linear_model import LinearRegression
from common import Config, evaluate, split_data

def run_step_five(report: pd.DataFrame) -> pd.DataFrame:
    """模型改进"""
    print("=" * 25)

    x_train, x_test, y_train, y_test = split_data()
    selected =
x_train.corrwith(y_train).abs().nlargest(3).index.tolist()
    model = LinearRegression().fit(x_train[selected], y_train)
    row = [
        "三个特征回归",
```

```
        len(selected),
        *evaluate(y_train, model.predict(x_train[selected])),
        *evaluate(y_test, model.predict(x_test[selected])),
    ]
    report = pd.concat(
        [report, pd.DataFrame([row],
columns=Config.METRIC_COLUMNS)], ignore_index=True
    )

    print(f"选取特征: {'、'.join(selected)}")
    print("模型汇总: \n" + report.round(4).to_string(index=False))

    return report
```